Foto de famoso:



```
)

# ── Mostrar lo que ha descubierto ──
for cara in resultados:
    print("═══════════════════════════════")
    print(f"😊 Emoción detectada: {cara['dominant_emotion']}")
    print(f"🎂 Edad estimada:     {cara['age']} años")
    print(f"👤 Género:            {cara['dominant_gender']}")
    print(f"📍 Posición cara:     x={cara['region']['x']}, y={cara['region']['y']}")
    print(f"📐 Tamaño cara:       {cara['region']['w']}×{cara['region']['h']} píxeles")
    print(f"🌍 Etnia: {cara['dominant_race']}")
    print("═══════════════════════════════")
```

```
Action: race: 100%|██████████| 4/4 [00:02<00:00,  1.59it/s]═══════════════════════════
😊 Emoción detectada: neutral
🎂 Edad estimada:     57 años
👤 Género:            Man
📍 Posición cara:     x=677, y=105
📐 Tamaño cara:       259×372 píxeles
🌍 Etnia: middle eastern
═══════════════════════════════
```

Foto mia:



```
from deepface import DeepFace

# ── Analizar la foto ──
# Cambia "foto.jpg" por el nombre de tu foto subida
resultados = DeepFace.analyze(
    img_path="WIN_20260915_18_34_59_Pro.jpg",
    actions=["age", "gender", "emotion", "race"] ,
    detector_backend= "mtcnn"
)

# ── Mostrar lo que ha descubierto ──
for cara in resultados:
    print("═══════════════════════════════")
    print(f"😊 Emoción detectada: {cara['dominant_emotion']}")
    print(f"🎂 Edad estimada:     {cara['age']} años")
    print(f"👤 Género:            {cara['dominant_gender']}")
    print(f"📍 Posición cara:     x={cara['region']['x']}, y={cara['region']['y']}")
    print(f"📐 Tamaño cara:       {cara['region']['w']}×{cara['region']['h']} píxeles")
    print(f"🌍 Etnia: {cara['dominant_race']}")
    print("═══════════════════════════════")
```

```
Action: race: 100%|██████████| 4/4 [00:02<00:00,  1.78it/s]═══════════════════════════
😊 Emoción detectada: neutral
🎂 Edad estimada:     27 años
👤 Género:            Man
📍 Posición cara:     x=932, y=311
📐 Tamaño cara:       179×223 píxeles
🌍 Etnia: white
═══════════════════════════════
```

Foto de un familiar:

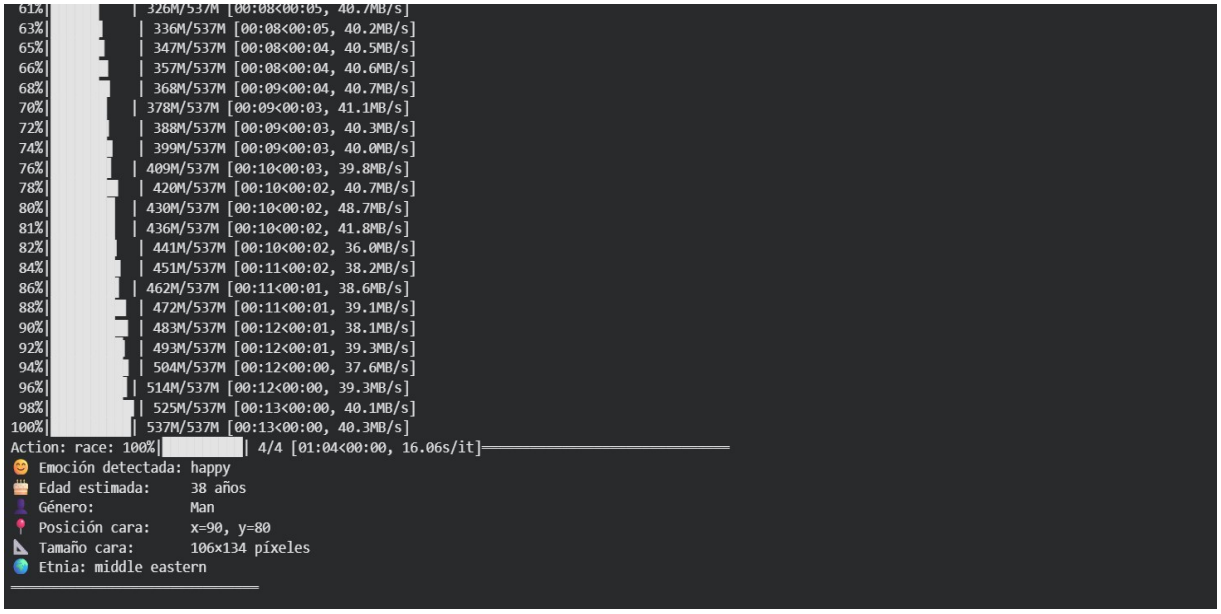


Tabla final con resultados:

FOTO	EMOCION	EDAD REAL	EDAD ESTIMADA	¿ACIERTA?
Yo	neutral	16	27	En la emocion si
Familiar	happy	40	38	Se aproxima bastante a la realidad
Famoso	neutral	66	57	En la emoción si

Reflexión:

Acierta más en el caso de un familiar.
Falla más en en el caso de mi foto y de un famoso,ya que falla en lo mismo.
Sí, la calidad de la foto influye bastante.

```python
from deepface import DeepFace

# ── Buscar todas las caras en la imagen ──
caras = DeepFace.extract_faces(
    img_path="artworks-VqW5oZ7OCPKyA9ny-WX1DzQ-t500x500.jpg",
-   detector_backend="opencv",  # El detector mas rapido
+   detector_backend="mtcnn",  # El detector mas rapido
    align=True                  # Alinear caras (ojos horizontales)
)

# ── Mostrar cuántas caras encontró ──
print(f"🔍 Caras encontradas: {len(caras)}")

# ── Detalles de cada cara ──
for numero, cara in enumerate(caras, start=1):
    zona = cara["facial_area"]      # Coordenadas de la cara
    confianza = cara["confidence"]  # Seguridad de la detección (0 a 1)

    print(f"\n👤 Cara {numero}:")
    print(f"   Posición: x={zona['x']}, y={zona['y']}")
    print(f"   Tamaño:   {zona['w']}×{zona['h']} píxeles")
    print(f"   Confianza: {confianza:.2%}")
```

```
🔍 Caras encontradas: 6

👤 Cara 1:
   Posición: x=126, y=83
   Tamaño:   100×126 píxeles
   Confianza: 100.00%

👤 Cara 2:
   Posición: x=398, y=99
   Tamaño:   101×138 píxeles
   Confianza: 100.00%

👤 Cara 3:
   Posición: x=0, y=153
   Tamaño:   111×160 píxeles
   Confianza: 100.00%

👤 Cara 4:
   Posición: x=297, y=237
   Tamaño:   138×190 píxeles
   Confianza: 100.00%

👤 Cara 5:
   Posición: x=71, y=241
   Tamaño:   166×230 píxeles
   Confianza: 100.00%

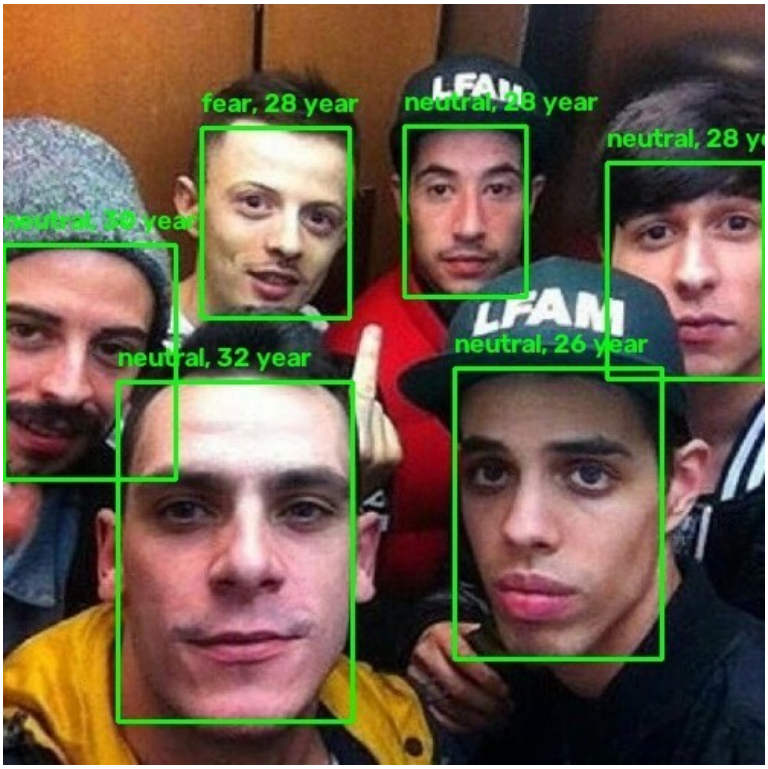
👤 Cara 6:
   Posición: x=264, y=79
   Tamaño:   79×113 píxeles
   Confianza: 99.00%
```

Uso mtcnn por que opencv no me funciona, y si detecta todas las caras.



Si uso retinaface, tarda un poco más, pero el resultado es el mismo.

Actividad 5:

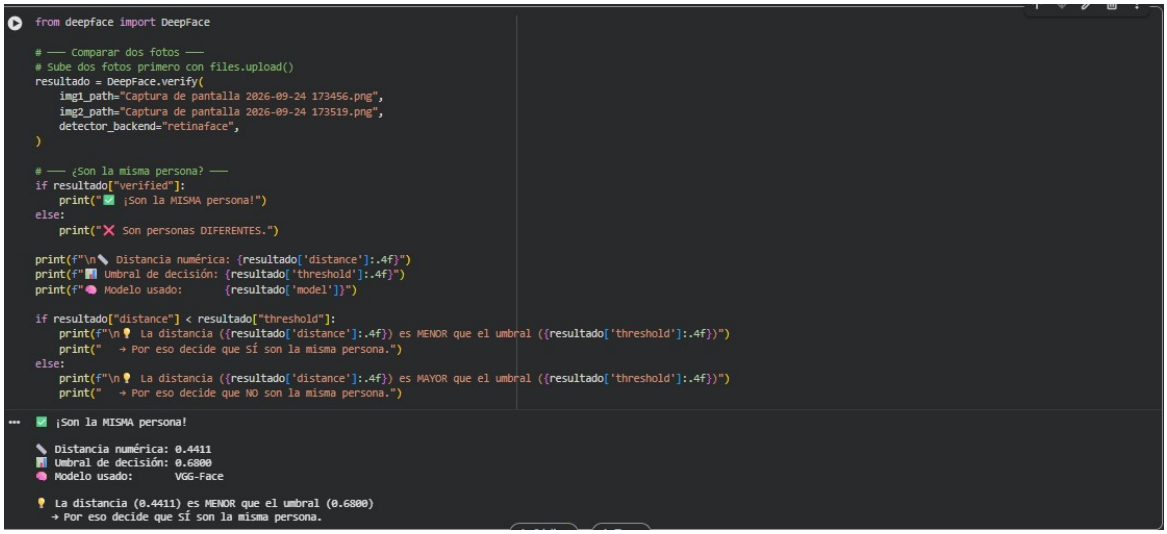


Actividad 5.1-2:



Actividad 6.1:

1. Dos fotos tuyas (hechas en momentos diferentes). ¿Dice que eres tú?



```
from deepface import DeepFace

# ── Comparar dos fotos ──
# Sube dos fotos primero con files.upload()
resultado = DeepFace.verify(
    img1_path="Captura de pantalla 2026-09-24 173456.png",
    img2_path="Captura de pantalla 2026-09-24 173519.png",
    detector_backend="retinaface",
)

# ── ¿Son la misma persona? ──
if resultado["verified"]:
    print("✅ ¡Son la MISMA persona!")
else:
    print("❌ Son personas DIFERENTES.")

print(f"\n📏 Distancia numérica: {resultado['distance']:.4f}")
print(f"📊 Umbral de decisión: {resultado['threshold']:.4f}")
print(f"🧠 Modelo usado:       {resultado['model']}")

if resultado["distance"] < resultado["threshold"]:
    print(f"\n💡 La distancia ({resultado['distance']:.4f}) es MENOR que el umbral ({resultado['threshold']:.4f})")
    print("   → Por eso decide que SÍ son la misma persona.")
else:
    print(f"\n💡 La distancia ({resultado['distance']:.4f}) es MAYOR que el umbral ({resultado['threshold']:.4f})")
    print("   → Por eso decide que NO son la misma persona.")
```

```
✅ ¡Son la MISMA persona!

📏 Distancia numérica: 0.4411
📊 Umbral de decisión: 0.6800
🧠 Modelo usado:       VGG-Face

💡 La distancia (0.4411) es MENOR que el umbral (0.6800)
   → Por eso decide que SÍ son la misma persona.
```

Si dice que soy yo.

2. Una foto tuya y una de un amigo/a. ¿Dice que sois diferentes?



```
from deepface import DeepFace

# ── Comparar dos fotos ──
# Sube dos fotos primero con files.upload()
resultado = DeepFace.verify(
    img1_path="Captura de pantalla 2026-09-24 173456.png",
    img2_path="Captura de pantalla 2026-09-24 174046.png",
    detector_backend="retinaface",
)

# ── ¿Son la misma persona? ──
if resultado["verified"]:
    print("✅ ¡Son la MISMA persona!")
else:
    print("❌ Son personas DIFERENTES.")

print(f"\n📏 Distancia numérica: {resultado['distance']:.4f}")
print(f"📊 Umbral de decisión: {resultado['threshold']:.4f}")
print(f"🧠 Modelo usado:       {resultado['model']}")

if resultado["distance"] < resultado["threshold"]:
    print(f"\n💡 La distancia ({resultado['distance']:.4f}) es MENOR que el umbral ({resultado['threshold']:.4f})")
    print("   → Por eso decide que SÍ son la misma persona.")
else:
    print(f"\n💡 La distancia ({resultado['distance']:.4f}) es MAYOR que el umbral ({resultado['threshold']:.4f})")
    print("   → Por eso decide que NO son la misma persona.")
```

```
❌ Son personas DIFERENTES.

📏 Distancia numérica: 0.7759
📊 Umbral de decisión: 0.6800
🧠 Modelo usado:       VGG-Face

💡 La distancia (0.7759) es MAYOR que el umbral (0.6800)
   → Por eso decide que NO son la misma persona.
```

Dice que somos diferentes

3. Dos fotos de la misma persona pero una con gafas de sol. ¿Funciona?



```
from deepface import DeepFace

# ── Comparar dos fotos ──
# Sube dos fotos primero con files.upload()
resultado = DeepFace.verify(
    img1_path="images (6).jpg",
    img2_path="Leo_Messi_Argentina_v_Egypt_7_July_2026-1.jpg",
    detector_backend="retinaface",
)

# ── ¿Son la misma persona? ──
if resultado["verified"]:
    print("✅ ¡Son la MISMA persona!")
else:
    print("❌ Son personas DIFERENTES.")

print(f"\n📏 Distancia numérica: {resultado['distance']:.4f}")
print(f"📊 Umbral de decisión: {resultado['threshold']:.4f}")
print(f"🧠 Modelo usado:       {resultado['model']}")

if resultado["distance"] < resultado["threshold"]:
    print(f"\n💡 La distancia ({resultado['distance']:.4f}) es MENOR que el umbral ({resultado['threshold']:.4f})")
    print("   → Por eso decide que SÍ son la misma persona.")
else:
    print(f"\n💡 La distancia ({resultado['distance']:.4f}) es MAYOR que el umbral ({resultado['threshold']:.4f})")
    print("   → Por eso decide que NO son la misma persona.")
```

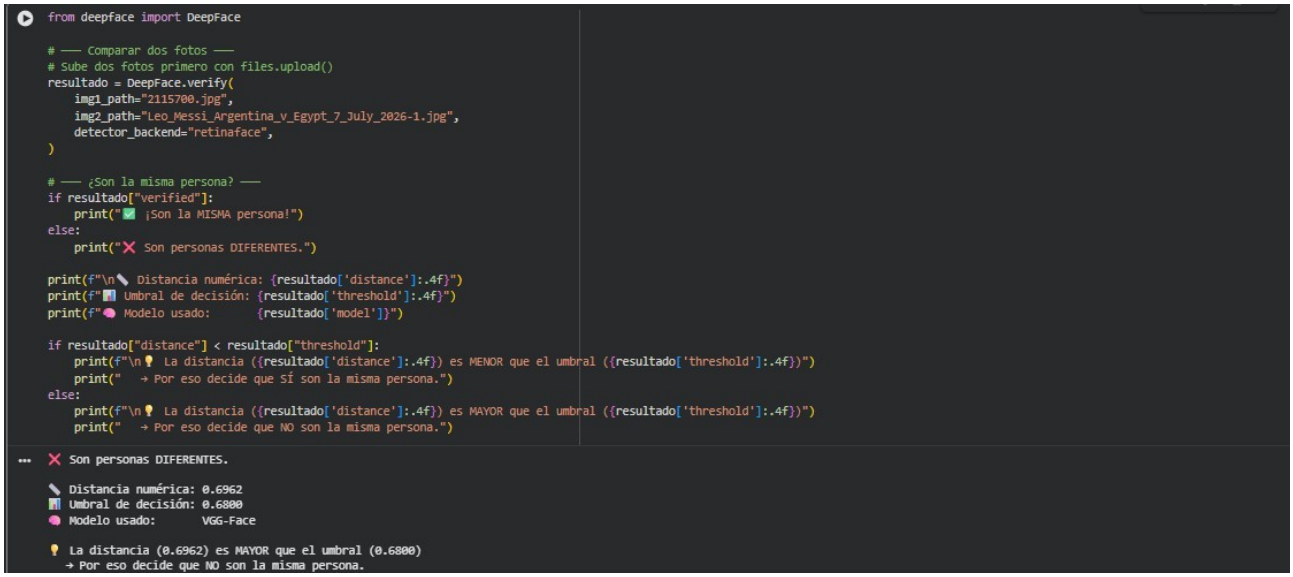
```
✅ ¡Son la MISMA persona!

📏 Distancia numérica: 0.6474
📊 Umbral de decisión: 0.6800
🧠 Modelo usado:       VGG-Face

💡 La distancia (0.6474) es MENOR que el umbral (0.6800)
   → Por eso decide que SÍ son la misma persona.
```

Dice que es la misma persona.

4. Una foto de una persona de frente y otra de perfil. ¿Qué pasa?



```
from deepface import DeepFace

# ── Comparar dos fotos ──
# Sube dos fotos primero con files.upload()
resultado = DeepFace.verify(
    img1_path="2115700.jpg",
    img2_path="Leo_Messi_Argentina_v_Egypt_7_July_2026-1.jpg",
    detector_backend="retinaface",
)

# ── ¿Son la misma persona? ──
if resultado["verified"]:
    print("✅ ¡Son la MISMA persona!")
else:
    print("❌ Son personas DIFERENTES.")

print(f"\n📏 Distancia numérica: {resultado['distance']:.4f}")
print(f"📊 Umbral de decisión: {resultado['threshold']:.4f}")
print(f"🧠 Modelo usado:       {resultado['model']}")

if resultado["distance"] < resultado["threshold"]:
    print(f"\n💡 La distancia ({resultado['distance']:.4f}) es MENOR que el umbral ({resultado['threshold']:.4f})")
    print("   → Por eso decide que SÍ son la misma persona.")
else:
    print(f"\n💡 La distancia ({resultado['distance']:.4f}) es MAYOR que el umbral ({resultado['threshold']:.4f})")
    print("   → Por eso decide que NO son la misma persona.")
```

```
❌ Son personas DIFERENTES.

📏 Distancia numérica: 0.6962
📊 Umbral de decisión: 0.6800
🧠 Modelo usado:       VGG-Face

💡 La distancia (0.6962) es MAYOR que el umbral (0.6800)
   → Por eso decide que NO son la misma persona.
```

Dice que son diferentes personas.
Resultados:
1. 0.4411 **Misma persona**
2. 0.6474 **Misma persona**
3. 0.6962 **Diferentes**
4. 0.7759 **Diferentes**

Los fallos aparecen cuando la distancia está muy cerca del umbral (0.68)

Actividad 7.1 — Emociones actuadas

Cara de sorprendido:

Probabilidad de cada emocion:

angry: 0.0%
disgust: 0.0%
fear: [redacted] 65.9%
happy: 0.1%
sad: 0.0%
surprise: [redacted] 34.0%
neutral: 0.0%

Cara de tristeza:

Probabilidad de cada emocion:

angry: [redacted] 88.4%
disgust: 0.0%
fear: 1.7%
happy: 0.0%
sad: [redacted] 9.6%
surprise: 0.0%
neutral: 0.3%

Cara de felicidad:
Probabilidad de cada emocion:
angry: 1.1%
disgust: 0.0%
fear: 0.0%
happy: 0.0%
sad: 3.8%
surprise: 0.0%
neutral: 95.0%

Cara de enfadado:
Probabilidad de cada emocion:
angry: 6.0%
disgust: 0.0%
fear: 0.0%
happy: 0.0%
sad: 0.2%
surprise: 0.0%
neutral: 93.8%

La de sorprendido es la que más se asemeja, confunde todas, posiblemente por la luz del entorno.

Actividad 8.1 — Tu propia base de datos

Base de datos:



```
import os
from google.colab import files

# ── Crear la carpeta de la base de datos ──
os.makedirs("base_datos", exist_ok=True)

# ── Subir fotos de personas conocidas ──
print("📸 Sube las fotos de las personas conocidas.")
print("   El NOMBRE del archivo será el nombre de la persona.")
print("   Ejemplo: Ana.jpg, Carlos.jpg, Lucia.png\n")

subidas = files.upload()

# ── Mover las fotos a la carpeta ──
for nombre_archivo in subidas:
    ruta_destino = os.path.join("base_datos", nombre_archivo)
    os.rename(nombre_archivo, ruta_destino)
    nombre_persona = os.path.splitext(nombre_archivo)[0]
    print(f"  ✅ {nombre_persona} registrado/a")

print(f"\n🎉 Base de datos lista: {len(subidas)} persona(s)")
```

```
📸 Sube las fotos de las personas conocidas.
   El NOMBRE del archivo será el nombre de la persona.
   Ejemplo: Ana.jpg, Carlos.jpg, Lucia.png

Elegir archivos  3 archivos
Ruben.png(image/png) - 268935 bytes, last modified: 24/9/2026 - 100% done
Carlos.png(image/png) - 285768 bytes, last modified: 24/9/2026 - 100% done
Messi.jpg(image/jpeg) - 465966 bytes, last modified: 24/9/2026 - 100% done
Saving Ruben.png to Ruben.png
Saving Carlos.png to Carlos.png
Saving Messi.jpg to Messi.jpg
  ✅ Ruben registrado/a
  ✅ Carlos registrado/a
  ✅ Messi registrado/a

🎉 Base de datos lista: 3 persona(s)
```

Foto de messi:

```
from deepface import DeepFace
from google.colab import files
import os

# ── Subir la foto de la persona a identificar ──
print("📷 Sube la foto de la persona que quieres identificar:\n")
subida = files.upload()
foto_desconocida = list(subida.keys())[0]

# ── Buscar en la base de datos ──
print(f"\n🔍 Buscando coincidencias para '{foto_desconocida}'...\n")

resultados = DeepFace.find(
    img_path=foto_desconocida,
    db_path="base_datos",
    detector_backend="retinaface",
    enforce_detection=False,
    silent=True,
    refresh_database=True
)

# ── Interpretar el resultado ──
if len(resultados) > 0 and len(resultados[0]) > 0:
    mejor_coincidencia = resultados[0].iloc[0]
    ruta_foto = mejor_coincidencia["identity"]
    distancia = mejor_coincidencia["distance"]

    # Extraer el nombre del archivo (sin extensión ni carpeta)
    nombre = os.path.splitext(os.path.basename(ruta_foto))[0]

    print(f"🎯 Persona identificada: {nombre}")
    print(f"📏 Distancia: {distancia:.4f}")
    print(f"📄 Foto coincidente: {ruta_foto}")
else:
    print("❓ No se encontró coincidencia en la base de datos.")
    print("   Puede ser una persona no registrada o la foto no es clara.")
```

```
📷 Sube la foto de la persona que quieres identificar:

Elegir archivos  messi67.jpg
messi67.jpg(image/jpeg) - 20202 bytes, last modified: 24/9/2026 - 100% done
Saving messi67.jpg to messi67 (1).jpg

🔍 Buscando coincidencias para 'messi67 (1).jpg'...

❓ No se encontró coincidencia en la base de datos.
   Puede ser una persona no registrada o la foto no es clara.
```

Foto de Rubén:

```
from deepface import DeepFace
from google.colab import files
import os

# ── Subir la foto de la persona a identificar ──
print("📷 Sube la foto de la persona que quieres identificar:\n")
subida = files.upload()
foto_desconocida = list(subida.keys())[0]

# ── Buscar en la base de datos ──
print(f"\n🔍 Buscando coincidencias para '{foto_desconocida}'...\n")

resultados = DeepFace.find(
    img_path=foto_desconocida,
    db_path="base_datos",
    detector_backend="retinaface",
    enforce_detection=False,
    silent=True,
    refresh_database=True
)

# ── Interpretar el resultado ──
if len(resultados) > 0 and len(resultados[0]) > 0:
    mejor_coincidencia = resultados[0].iloc[0]
    ruta_foto = mejor_coincidencia["identity"]
    distancia = mejor_coincidencia["distance"]

    # Extraer el nombre del archivo (sin extensión ni carpeta)
    nombre = os.path.splitext(os.path.basename(ruta_foto))[0]

    print(f"🎯 Persona identificada: {nombre}")
    print(f"📏 Distancia: {distancia:.4f}")
    print(f"📄 Foto coincidente: {ruta_foto}")
else:
    print("❓ No se encontró coincidencia en la base de datos.")
    print("   Puede ser una persona no registrada o la foto no es clara.")
```

```
📷 Sube la foto de la persona que quieres identificar:

Elegir archivos  Ruben1.png
Ruben1.png(image/png) - 203103 bytes, last modified: 24/9/2026 - 100% done
Saving Ruben1.png to Ruben1.png

🔍 Buscando coincidencias para 'Ruben1.png'...

🎯 Persona identificada: Ruben
📏 Distancia: 0.2943
📄 Foto coincidente: base_datos/Ruben.png
```

Foto de Carlos:

```python
from deepface import DeepFace
from google.colab import files
import os

# ── Subir la foto de la persona a identificar ──
print("📷 Sube la foto de la persona que quieres identificar:\n")
subida = files.upload()
foto_desconocida = list(subida.keys())[0]

# ── Buscar en la base de datos ──
print(f"\n🔍 Buscando coincidencias para '{foto_desconocida}'...\n")

resultados = DeepFace.find(
    img_path=foto_desconocida,
    db_path="base_datos",
    detector_backend="retinaface",
    enforce_detection=False,
    silent=True,
    refresh_database=True
)

# ── Interpretar el resultado ──
if len(resultados) > 0 and len(resultados[0]) > 0:
    mejor_coincidencia = resultados[0].iloc[0]
    ruta_foto = mejor_coincidencia["identity"]
    distancia = mejor_coincidencia["distance"]

    # Extraer el nombre del archivo (sin extensión ni carpeta)
    nombre = os.path.splitext(os.path.basename(ruta_foto))[0]

    print(f"🎯 Persona identificada: {nombre}")
    print(f"📏 Distancia: {distancia:.4f}")
    print(f"📄 Foto coincidente: {ruta_foto}")
else:
    print("❓ No se encontró coincidencia en la base de datos.")
    print("   Puede ser una persona no registrada o la foto no es clara.")
```

```
📷 Sube la foto de la persona que quieres identificar:

Elegir archivos  Carlos1.png
Carlos1.png(image/png) - 373069 bytes, last modified: 24/9/2026 - 100% done
Saving Carlos1.png to Carlos1.png

🔍 Buscando coincidencias para 'Carlos1.png'...

🎯 Persona identificada: Carlos
📏 Distancia: 0.4504
📄 Foto coincidente: base_datos/Carlos.png
```

Identifica correctamente las fotos de Ruben y Carlos, en cambio, la de Messi, no se por que no la reconoce, en la de Rubén, da una distancia de 0.2943 u, y en Carlos, sale 0.4504 u, poniendo el ejemplo de Messi, sale que no encuentra incidencia en la base de datos.

Camino B: Solo navegador:

FÁCIL **Actividad 9b.1 — Checklist** FÁCIL

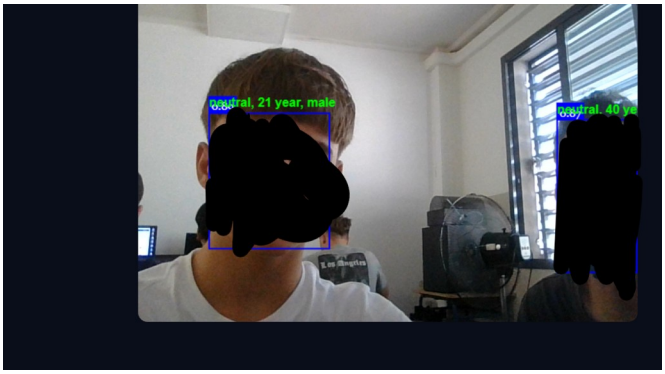
Marca cada punto cuando lo tengas:

- ☐ Carpeta `reconocimiento_web` creada
- ☐ Archivo `test.html` creado y abierto con el Bloc de notas
- ☐ Codigo copiado y guardado
- ☐ Al abrir en Chrome, muestra la versión de face-api.js

Todo hecho.

Actividad 10b.1 — Prueba la webcam

Sí, aparece el rectángulo, pero la emoción falla un poco, sí, detecta todas las caras

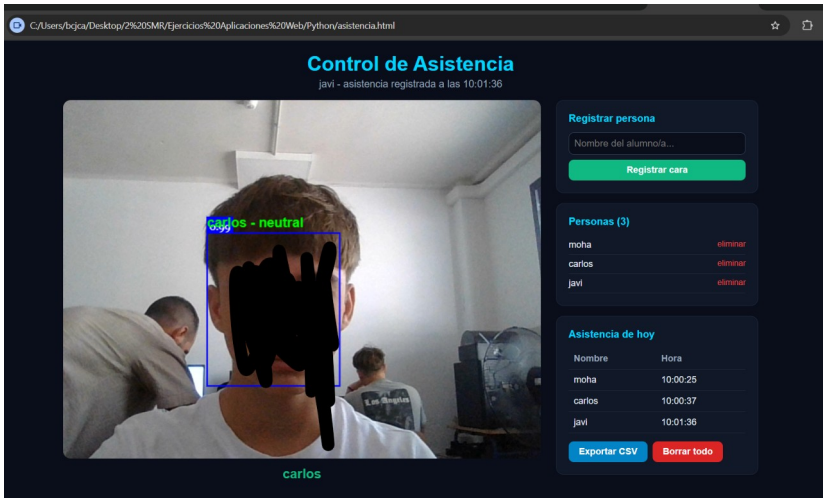


Actividad 10b.2 — Personaliza

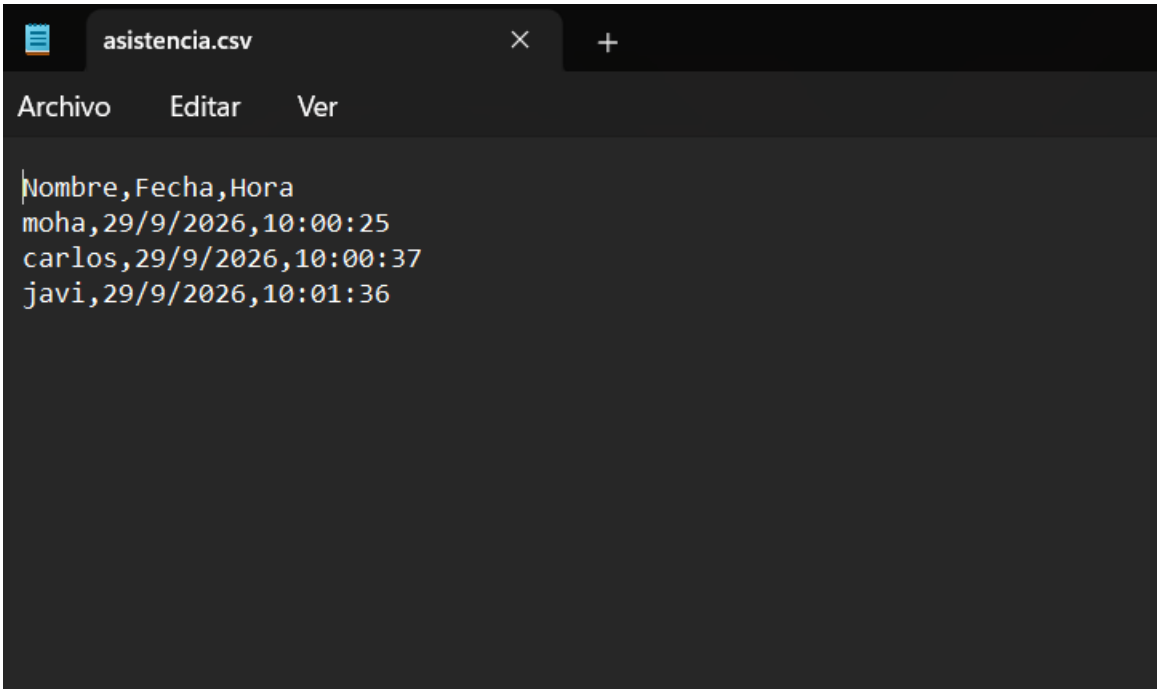


Al cambiar el intervalo a 200, el vídeo va más rapido. Al cambiar el tamaño de letra, se ve un poco mejor.

11b. Proyecto: Asistencia en el navegador



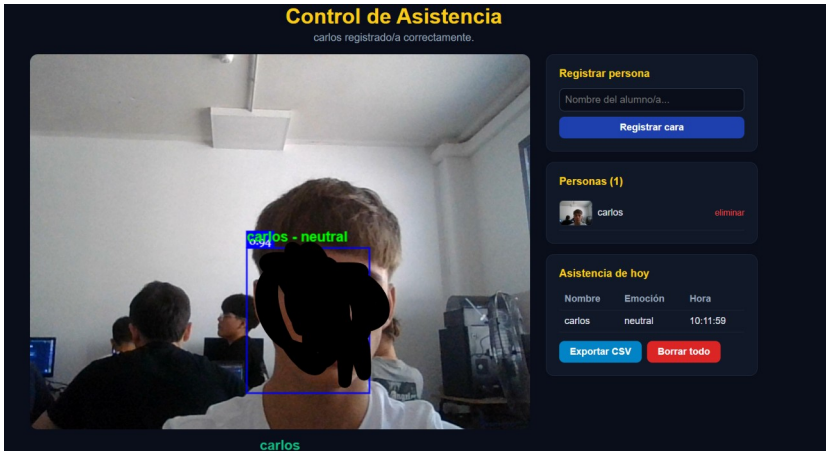
Lo identifica y sale el cuadro en verde



Sí que registra los nombres y las horas en el archivo .csv

Al cerrar el .html y abrirlo otra vez, el registro sigue.

Reto 11b.2 — Mejora el sistema



Si pongo el umbral en 0.7, el sistema tiende más a fallos.

Camino A:

FÁCIL **Actividad 9.1 — Checklist de instalación** FÁCIL

Objetivo: Asegurarte de que todo está listo antes de empezar con la webcam.

Marca cada punto cuando lo tengas:

- ☐ `python --versión` muestra Python 3.10 o superior
- ☐ VS Code está instalado y se abre correctamente
- ☐ La extensión de Python esta instalada (icono de Extensiones → buscar "Python" → dice "Installed")
- ☐ Has abierto la carpeta `reconocimiento_facial` en VS Code
- ☐ El archivo `test_deepface.py` se ejecuta sin errores
- ☐ El terminal muestra las versiónes de DeepFace y OpenCV

Si todos los puntos están marcados, estás listo para la **sección 10: Webcam en tiempo real**.

Todo hecho.

